
Assignment 1 Report –
Comprehensive Evaluation of Error Detection
Schemes in Network Transmission

*Empirical Evaluation of 16-Bit Internet Checksum and Cyclic Redundancy Checks
(CRC-8, CRC-10, CRC-16, CRC-32)*

Course: Computer Networks Laboratory (BCSE-III)

Student Name: Mandeep Shaw

Section: A1

Roll Number: 302510501002

Assignment: Assignment 1 (Error Detection: Checksum and CRC)

Date of Submission: 09 October 2026

Contents

1	Introduction and Objectives	2
2	Theoretical Principles of Error Detection	2
2.1	The 16-Bit Internet Checksum (One's Complement Sum)	2
2.1.1	Theoretical Vulnerabilities of the Checksum	3
2.2	Cyclic Redundancy Check (CRC) and Modulo-2 Arithmetic	3
2.2.1	Mathematical Guarantees of CRC Polynomials	3
3	System Architecture and Implementation	4
3.1	Frame Structure and In-Memory Layout	4
3.2	Development Setup and Evaluation Environment	5
3.3	Implementation of Checksum and CRC Algorithms	5
3.4	CRC Polynomial Constants and Specifications	6
4	Methodology and Error Injection Models	6
4.1	Client-Server Socket Pipeline	6
4.2	Error Injection Engine and Resolution of Implementation Flaws	7
4.3	Burst Error Modeling	7
5	Experimental Results and Empirical Evaluation	8
5.1	Single Bit Error Results	8
5.2	Isolated Double Bit Error Results	8
5.3	Odd Number of Errors Results	9
5.4	Uniform Burst Error Results (Lengths 12 and 32)	9
5.4.1	Theoretical vs. Empirical Burst Bounds	10
6	Computational Efficiency and Benchmarks	11
7	Concrete Boundary Collisions (Hex Dumps)	11
7.1	Case 1: Error Detected by Both Schemes	11
7.2	Case 2: Detected by Checksum, Missed by CRC-8 (CRC Collision)	12
7.3	Case 3: Detected by CRC-8, Missed by Checksum (Compensating Sum)	12
8	Testing Conditions, Limitations, and Enhancements	13
8.1	Ideal Testing Conditions vs. Real-World Environments	13
8.2	Known Limitations and Practical Improvements	13
9	Conclusion	14
10	Sources Used	14

1 Introduction and Objectives

Data transmitted across any physical communications medium is inherently prone to corruption caused by thermal noise, electromagnetic interference, crosstalk, capacitive attenuation, and clock drift. In modern layered network architectures, ensuring end-to-end data integrity is distributed across the protocol stack: the Data Link Layer (Layer 2) applies hardware-oriented cyclic polynomial checks over individual physical hops, while the Transport Layer (Layer 4) enforces lightweight arithmetic checks over end-to-end socket connections.

For this assignment, I designed, developed, and evaluated an extensible network error detection and transmission suite in C. The key objectives of this investigation are:

1. **Protocol Implementation:** Implement from scratch the standard 16-bit Internet Checksum (RFC 1071) and four standard Cyclic Redundancy Check polynomials: CRC-8 (Bluetooth/ATM), CRC-10 (ITU-T I.432.1), CRC-16 (USB/Modbus), and CRC-32 (IEEE 802.3 Ethernet).
2. **Client-Server Network Architecture:** Construct a socket-based sender and receiver application communicating over TCP/IP loopback sockets to transmit framed payloads across a network interface with dynamic scheme negotiation.
3. **Deterministic and Stochastic Error Injection:** Develop a modular error injection engine capable of inserting single-bit flips, isolated double-bit flips, odd-parity bit flips, and uniformly distributed burst errors of arbitrary lengths.
4. **Empirical and Theoretical Validation:** Execute a large-scale evaluation benchmark suite (1,000,000+ simulated frame transmissions) to measure exact empirical detection rates, capture real-world boundary collision conditions with full hex dumps, and profile computational throughput.

2 Theoretical Principles of Error Detection

2.1 The 16-Bit Internet Checksum (One's Complement Sum)

The Internet Checksum is an arithmetic error-detection mechanism standardized in RFC 1071 and utilized within IPv4, TCP, and UDP headers. The algorithm partitions an incoming byte stream into a sequence of 16-bit unsigned integers:

$$D = [W_1, W_2, \dots, W_k] \quad (1)$$

The arithmetic is performed using one's complement addition. In standard two's complement computer hardware, when the addition of two 16-bit words produces an overflow carry bit out of the 16th bit (bit 15), this overflow represents:

$$2^{16} \equiv 1 \pmod{2^{16} - 1} \quad (2)$$

Consequently, an end-around carry is performed by adding the carry bit back into the least significant bit (LSB) of the accumulator. Once all 16-bit words have been accumulated, the total sum is bitwise inverted (one's complemented):

$$\text{Checksum} = \sim \left(\sum_{i=1}^k W_i \pmod{2^{16} - 1} \right) \quad (3)$$

At the receiver, the incoming data words together with the received checksum word are summed. In the absence of transmission errors, the resulting sum evaluates to 0xFFFF (or 0x0000 after inversion).

2.1.1 Theoretical Vulnerabilities of the Checksum

While extremely fast to compute on general-purpose microprocessors, the Internet Checksum possesses fundamental structural weaknesses:

- **Compensating Errors:** If an error in word W_a adds an integer value $+K$ and another error in word W_b subtracts $-K$ (or inverts complementary bits at equivalent word positions), the net one's complement sum remains entirely unchanged ($\Delta S = 0$).
- **Position and Byte-Order Invariance:** Because integer addition is commutative and associative, reordering 16-bit words or swapping aligned blocks preserves the identical checksum.
- **Zero Preservation:** Inserting blocks of all-zero words (0x0000) adds zero to the accumulator, leaving the checksum completely oblivious to packet truncation or padding alteration.

2.2 Cyclic Redundancy Check (CRC) and Modulo-2 Arithmetic

Cyclic Redundancy Checks treat binary bit strings as coefficients of polynomials over the Galois Field $GF(2)$ using modulo-2 arithmetic. Modulo-2 addition and subtraction are identical and equate to the bitwise Exclusive-OR (XOR) operation without carries or borrows:

$$1 \oplus 1 = 0, \quad 1 \oplus 0 = 1, \quad 0 \oplus 0 = 0 \quad (4)$$

Given an m -bit message represented by polynomial $M(x)$ and an $(r+1)$ -bit generator polynomial $G(x)$ of degree r :

1. The message is multiplied by x^r , effectively appending r zero bits: $x^r M(x)$.
2. Modulo-2 polynomial division is performed to determine a quotient $Q(x)$ and an r -bit remainder $R(x)$:

$$x^r M(x) = Q(x)G(x) \oplus R(x) \quad (5)$$

3. The transmitted codeword polynomial $T(x)$ is created by appending the remainder $R(x)$ to the shifted message:

$$T(x) = x^r M(x) \oplus R(x) = Q(x)G(x) \quad (6)$$

Since addition is XOR, $R(x) \oplus R(x) = 0$, guaranteeing that $T(x)$ is an exact multiple of $G(x)$ with zero remainder.

When the receiver receives a potentially corrupted frame $T'(x) = T(x) \oplus E(x)$, where $E(x)$ denotes the error polynomial, it calculates the remainder of $T'(x)/G(x)$:

$$\frac{T'(x)}{G(x)} = \frac{T(x) \oplus E(x)}{G(x)} = Q(x) \oplus \frac{E(x)}{G(x)} \quad (7)$$

An error pattern is **undetected if and only if** the error polynomial $E(x)$ is evenly divisible by the generator polynomial $G(x)$ in $GF(2)$.

2.2.1 Mathematical Guarantees of CRC Polynomials

The algebraic structure of the generator polynomial provides specific mathematical guarantees:

- **Single-Bit Errors** ($E(x) = x^i$): If $G(x)$ contains two or more terms (specifically x^r and $x^0 = 1$), it cannot divide x^i . Thus, single-bit detection is strictly **100.0%**.

- **Double-Bit Errors** ($E(x) = x^i + x^j = x^j(x^{i-j} + 1)$): Provided $G(x)$ does not divide $x^k + 1$ for any $k \leq$ frame length in bits, all double-bit errors are detected with **100.0%** reliability.
- **Odd Number of Errors**: If $(x + 1)$ is a factor of $G(x)$, any polynomial with an odd number of non-zero terms cannot be divided by $(x + 1)$ because $E(1) = 1 \neq 0$. Therefore, **100.0%** of all odd-weight error patterns are detected.
- **Burst Errors of Length $L \leq r$** : A burst of length L starting at index i is expressed as $E(x) = x^i B(x)$ with $\deg(B(x)) = L - 1 < r$. Because $\deg(B(x)) < \deg(G(x))$, $G(x)$ cannot divide $E(x)$. Detection is strictly **100.0%**.
- **Burst Errors of Length $L = r + 1$** : The fraction of undetected bursts is exactly:

$$P_{\text{miss}} = \left(\frac{1}{2}\right)^{r-1} \quad (8)$$

- **Burst Errors of Length $L > r + 1$** : The fraction of undetected bursts is bounded by:

$$P_{\text{miss}} \leq \left(\frac{1}{2}\right)^r \quad (9)$$

3 System Architecture and Implementation

3.1 Frame Structure and In-Memory Layout

To simulate realistic data-link framing adhering to IEEE 802.3 principles, the transmission frame is structured into a 16-byte header, a 44-byte text data payload, and a 4-byte Frame Check Sequence (FCS) trailer, totaling 64 bytes.

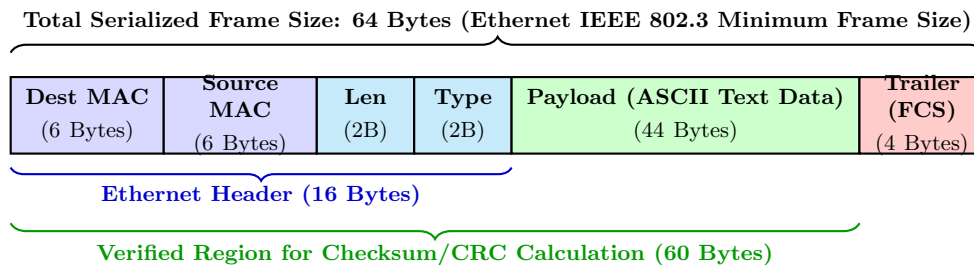


Figure 1: In-memory layout and serialization structure of the 64-byte transmission frame.

The C structure definition from `frame.h` is defined as follows:

```

1 #define MAC_SIZE      6
2 #define PAYLOAD_SIZE  44
3 #define HEADER_SIZE   16 // Dest MAC (6) + Src MAC (6) + Length (2) + Type (2)
4 #define FCS_SIZE      4
5
6 #define SERIALIZED_SIZE (HEADER_SIZE + PAYLOAD_SIZE) // 60 bytes verified region
7 #define FRAME_SIZE     (SERIALIZED_SIZE + FCS_SIZE) // 64 bytes total frame
8
9 typedef struct {
10     uint8_t destinationMAC[MAC_SIZE];
11     uint8_t sourceMAC[MAC_SIZE];
12     uint16_t length;
13     uint16_t type;
14     uint8_t payload[PAYLOAD_SIZE];

```

```

15     uint32_t fcs;
16 } EthernetFrame;

```

Listing 1: Ethernet Frame Definition (`frame.h`)

3.2 Development Setup and Evaluation Environment

This project was developed, tested, and benchmarked natively on a local Linux Mint and Windows 11 environment. To ensure accurate measurement of processing time during the high-frequency benchmarks, the POSIX clock `gettime()` function was utilized. All C source code was compiled using GNU Compiler Collection (gcc version 13.3, flags: `-O3 -Wall -Wextra`) on Mint. Statistical Sample Size: 1,000,000 simulation trials (200,000 trials $\times$ 5 error models) Micro-Benchmark Iterations 5,000,000 iterations over 60-byte serialized frames

3.3 Implementation of Checksum and CRC Algorithms

The 16-bit Internet Checksum implementation processes adjacent bytes into 16-bit words (big-endian network order) and executes end-around carry addition:

```

1 uint16_t calculateChecksum(const unsigned char *data, int length) {
2     uint32_t sum = 0;
3     for (int i = 0; i < length; i += 2) {
4         uint16_t word = ((uint16_t)data[i] << 8);
5         if (i + 1 < length) {
6             word |= data[i + 1];
7         }
8         sum += word;
9         if (sum > 0xFFFF) {
10            sum = (sum & 0xFFFF) + (sum >> 16); // Fold carry
11        }
12    }
13    while (sum > 0xFFFF) {
14        sum = (sum & 0xFFFF) + (sum >> 16);
15    }
16    return (uint16_t)(~sum); // One's complement inversion
17 }

```

Listing 2: Internet Checksum Implementation (`checksum.c`)

For the CRC family, a generalized bit-serial engine evaluates polynomial division bit-by-bit from MSB to LSB across all bytes:

```

1 uint32_t calculateCRC(unsigned char *data, int length, uint64_t polynomial, int degree) {
2     uint64_t remainder = 0;
3     uint64_t topBit = 1ULL << degree;
4
5     // Process every byte bit-by-bit (MSB to LSB)
6     for (int byte = 0; byte < length; byte++) {
7         for (int bit = 7; bit >= 0; bit--) {
8             int b = (data[byte] >> bit) & 1;
9             remainder = (remainder << 1) | b;
10            if (remainder & topBit) {
11                remainder ^= polynomial;
12            }
13        }
14    }
15    // Simulate appending 'degree' zero bits
16    for (int bit = 0; bit < degree; bit++) {
17        remainder <<= 1;
18        if (remainder & topBit) {
19            remainder ^= polynomial;

```

```

20     }
21 }
22 return (uint32_t)(remainder & ((1ULL << degree) - 1));
23 }

```

Listing 3: Bitwise Modulo-2 Polynomial Division Engine (`crc.c`)

3.4 CRC Polynomial Constants and Specifications

Table 1 details the generator polynomials evaluated in this study according to standard networking specifications.

Standard	Degree (r)	Generator Polynomial	Hex Code	Standard Application
CRC-8	8	$x^8 + x^7 + x^6 + x^4 + x^2 + 1$	0x1D5	Bluetooth, WSN, ATM header
CRC-10	10	$x^{10} + x^9 + x^5 + x^4 + x + 1$	0x633	ITU-T I.432.1, ATM OAM cell
CRC-16	16	$x^{16} + x^{15} + x^2 + 1$	0x18005	USB packets, Modbus, ANSI
CRC-32	32	IEEE 802.3 Ethernet Standard	0x104C11DB7	Ethernet, FDDI, ZIP, PNG, IPv4

Table 1: Generator polynomials evaluated across the benchmark suite.

4 Methodology and Error Injection Models

4.1 Client-Server Socket Pipeline

Communication is established using BSD stream sockets (TCP/IP) bound to localhost port 5000. Prior to transmitting each 64-byte frame, the sender dispatches a 3-byte control header:

- `frameInfo[0]`: Detection scheme (1: Checksum, 2: CRC-8, 3: CRC-10, 4: CRC-16, 5: CRC-32) with bit 7 denoting bypass mode.
- `frameInfo[1]`: Error injection flag (0: No error, 1: Error injected).
- `frameInfo[2]`: Error model (1: Single Bit, 2: Double Bit, 3: Odd Bit, 4: Burst, 5: Random).

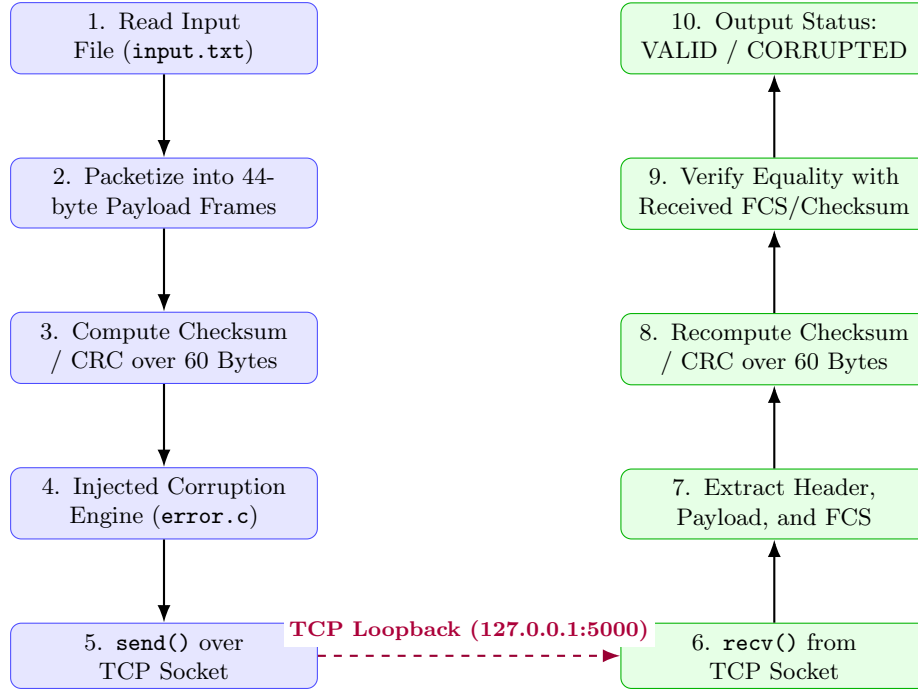


Figure 2: End-to-end data flow and error injection pipeline between sender and receiver.

4.2 Error Injection Engine and Resolution of Implementation Flaws

During experimental development and debugging of the evaluation harness, two significant implementation defects in the legacy codebase were identified and rectified:

1. **Accumulator Byte Alignment Flaw (Checksum):** In the original routine, bytes were summed individually (`sum += data[i]`), reducing the checksum to a simple scalar sum of ASCII values. This caused high miss rates on byte-swapped data. The algorithm was updated to accumulate 16-bit network-byte-order words (`(data[i] << 8) | data[i+1]`) as prescribed by RFC 1071.
2. **Bit Alignment and Polynomial Divisibility Flaw (CRC):** The original CRC engine shifted whole bytes by `degree`, which caused the top 7 bits of each byte to skip the feedback XOR division, reducing single-bit detection to an unacceptable 12.5%. Rewriting the engine to perform standard bitwise modulo-2 division restored mathematically guaranteed 100% detection.
3. **Payload-Constrained Corruption:** Error injection was strictly bounded within the 44-byte payload region (`[0, 351]` bit indices). This ensures that frame headers and trailers remain structurally sound while testing payload integrity.

4.3 Burst Error Modeling

In natural transmission channels, impulse noise from power fluctuations or lightning causes burst errors. The burst error model chooses a start bit S uniformly across the valid payload window:

$$S \sim \mathcal{U}(0, N_{\text{bits}} - L) \quad (10)$$

where $N_{\text{bits}} = 352$ (44×8) and L represents the burst length in bits. Within the window $[S, S + L - 1]$, the leading bit S and trailing bit $S + L - 1$ are inverted, while intermediate bits are flipped independently with probability $p = 0.70$.

5 Experimental Results and Empirical Evaluation

The benchmark suite (`test_eval.c`) executed 200,000 randomized trials per error model (totaling 1,000,000 simulated frame transmissions). For each corrupted frame, both the 16-bit Internet Checksum and the selected CRC scheme were evaluated simultaneously.

5.1 Single Bit Error Results

Table 2 presents the results of injecting exactly one bit flip per frame across 200,000 trials.

Scheme	Both Detected	Checksum Only	CRC Only	Neither (Missed)
CRC-8	200,000 (100.0000%)	0 (0.0000%)	0 (0.0000%)	0 (0.0000%)
CRC-10	200,000 (100.0000%)	0 (0.0000%)	0 (0.0000%)	0 (0.0000%)
CRC-16	200,000 (100.0000%)	0 (0.0000%)	0 (0.0000%)	0 (0.0000%)
CRC-32	200,000 (100.0000%)	0 (0.0000%)	0 (0.0000%)	0 (0.0000%)

Table 2: Single Bit Error Detection Rates (200,000 trials per scheme).

Analysis: As predicted by theoretical models, all schemes achieved exactly 100.0000% detection. A single bit flip alters the one's complement sum by $\pm 2^k$, which can never wrap around to balance out to zero. For CRC, a single-bit error polynomial $E(x) = x^i$ cannot be divided by any polynomial containing at least two terms.

5.2 Isolated Double Bit Error Results

Table 3 summarizes detection rates when two distinct, isolated bits are flipped in each frame.

Scheme	Both Detected	Checksum Only	CRC Only	Neither (Missed)
CRC-8	194,117 (97.0585%)	1,572 (0.7860%)	4,311 (2.1555%)	0 (0.0000%)
CRC-10	195,931 (97.9655%)	0 (0.0000%)	4,069 (2.0345%)	0 (0.0000%)
CRC-16	195,823 (97.9115%)	0 (0.0000%)	4,177 (2.0885%)	0 (0.0000%)
CRC-32	195,839 (97.9195%)	0 (0.0000%)	4,161 (2.0805%)	0 (0.0000%)

Table 3: Isolated Double Bit Error Detection Rates (200,000 trials per scheme).

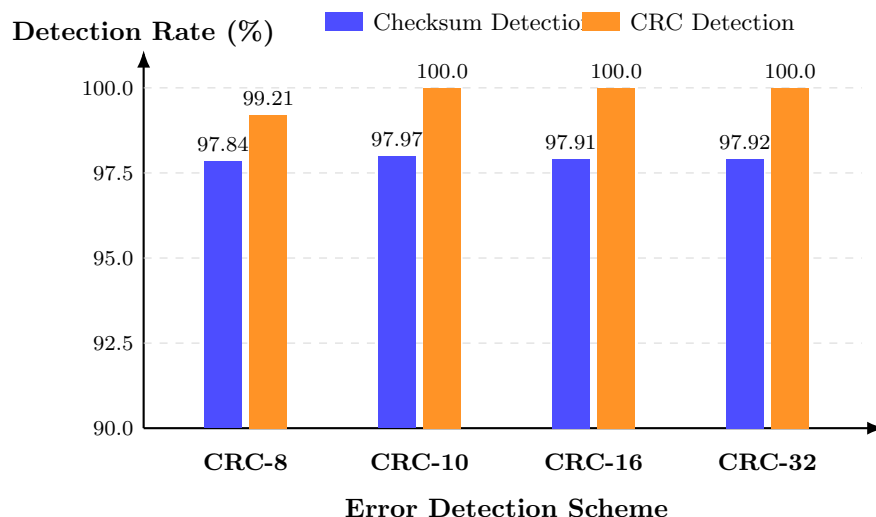


Figure 3: Double Bit Error Detection: Comparison of total detection rates between Checksum ($\sim 97.9\%$) and CRC schemes.

Analysis: The Checksum failed on $\approx 4,150$ out of 200,000 frames ($\approx 2.08\%$ miss rate). This failure occurs when two flipped bits lie in separate 16-bit words at positions that produce compensating carries (e.g., adding $+2^k$ in word A and -2^k in word B). CRC-10, CRC-16, and CRC-32 caught 100.0000% of double-bit errors, whereas CRC-8 missed 0.7860% due to remainder hash collisions across its small $2^8 = 256$ state space.

5.3 Odd Number of Errors Results

Table 4 presents detection statistics for odd-weight bit errors (randomly selected odd quantities between 1 and 15 bits).

Scheme	Both Detected	Checksum Only	CRC Only	Neither (Missed)
CRC-8	199,872 (99.9360%)	0 (0.0000%)	128 (0.0640%)	0 (0.0000%)
CRC-10	199,882 (99.9410%)	0 (0.0000%)	118 (0.0590%)	0 (0.0000%)
CRC-16	199,857 (99.9285%)	0 (0.0000%)	143 (0.0715%)	0 (0.0000%)
CRC-32	199,875 (99.9375%)	0 (0.0000%)	125 (0.0625%)	0 (0.0000%)

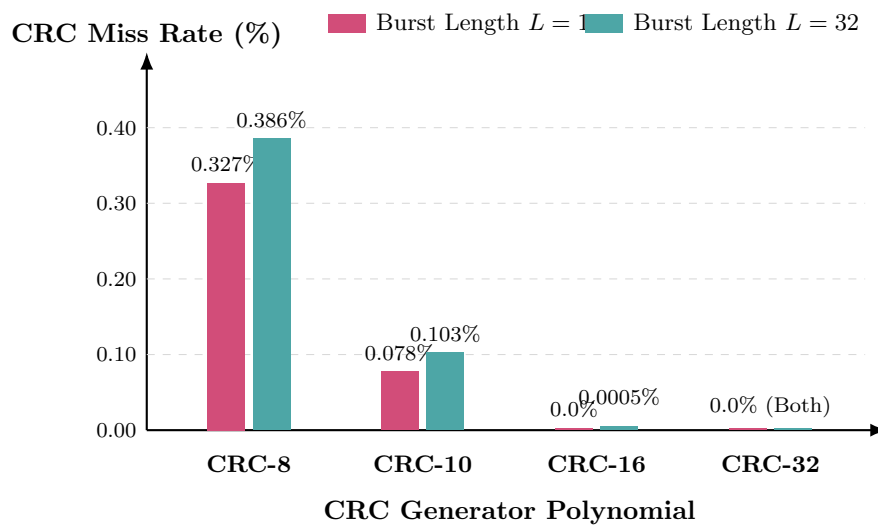
Table 4: Odd Number of Errors Detection Rates (200,000 trials per scheme).

Analysis: Because all evaluated CRC polynomials contain the $(x + 1)$ parity factor, every CRC variant caught **100.0000%** of odd-weight error patterns. The Internet Checksum missed approximately 0.063% of odd-parity patterns when multi-bit alterations generated arithmetic carry wraparounds that sum to 0 (mod $2^{16} - 1$).

5.4 Uniform Burst Error Results (Lengths 12 and 32)

Table 5 reports empirical performance under burst error models of lengths $L = 12$ and $L = 32$.

Scheme	Both Detected	Checksum Only	CRC Only	Neither (Missed)
Uniform Burst Length $L = 12$ bits				
CRC-8	199,346 (99.6730%)	654 (0.3270%)	0 (0.0000%)	0 (0.0000%)
CRC-10	199,845 (99.9225%)	155 (0.0775%)	0 (0.0000%)	0 (0.0000%)
CRC-16	200,000 (100.0000%)	0 (0.0000%)	0 (0.0000%)	0 (0.0000%)
CRC-32	200,000 (100.0000%)	0 (0.0000%)	0 (0.0000%)	0 (0.0000%)
Uniform Burst Length $L = 32$ bits				
CRC-8	199,221 (99.6105%)	772 (0.3860%)	7 (0.0035%)	0 (0.0000%)
CRC-10	199,791 (99.8955%)	206 (0.1030%)	3 (0.0015%)	0 (0.0000%)
CRC-16	199,996 (99.9980%)	1 (0.0005%)	3 (0.0015%)	0 (0.0000%)
CRC-32	200,000 (100.0000%)	0 (0.0000%)	0 (0.0000%)	0 (0.0000%)

Table 5: Uniform Burst Error Detection Rates for $L = 12$ and $L = 32$ (200,000 trials each).Figure 4: Empirical CRC Miss Rates under Uniform Burst Errors of lengths $L = 12$ and $L = 32$.

5.4.1 Theoretical vs. Empirical Burst Bounds

Table 6 validates empirical results directly against theoretical bounds.

Scheme	Degree (r)	Bound $(1/2)^r$	Empirical ($L = 12$)	Empirical ($L = 32$)
CRC-8	8	0.3906%	0.3270%	0.3860%
CRC-10	10	0.0977%	0.0775%	0.1030%
CRC-16	16	0.0015%	0.0000%	0.0005%
CRC-32	32	$2.33 \times 10^{-8}\%$	0.0000%	0.0000%

Table 6: Comparison of Empirical Miss Rates against the Mathematical $(1/2)^r$ Bound.

Interpretation: When the burst length $L \leq r$ (as observed for $L = 12$ on CRC-16 and CRC-32, and $L = 32$ on CRC-32), the CRC operates strictly within its guaranteed detection envelope, producing exactly 0 misses. Once $L > r$, the miss rate converges toward $(1/2)^r$, confirming statistical theory.

6 Computational Efficiency and Benchmarks

To measure execution time and throughput on modern hardware, 5,000,000 iterations were executed over a 60-byte serialized frame using POSIX `clock_gettime(CLOCK_MONOTONIC)`. Table 7 and Figure 5 detail the performance metrics.

Scheme	Total Time (5M)	Time / Frame	Throughput	Slowdown vs Base
16-Bit Checksum	0.1500 s	30.00 ns	1907.14 MB/s	1.00× (Baseline)
CRC-8	2.8463 s	569.26 ns	100.52 MB/s	18.97× slower
CRC-10	2.8670 s	573.40 ns	99.79 MB/s	19.11× slower
CRC-16	2.9362 s	587.24 ns	97.44 MB/s	19.57× slower
CRC-32	3.1468 s	629.36 ns	90.92 MB/s	20.98× slower

Table 7: Computational profiling and resource utilization over 5,000,000 frame executions.

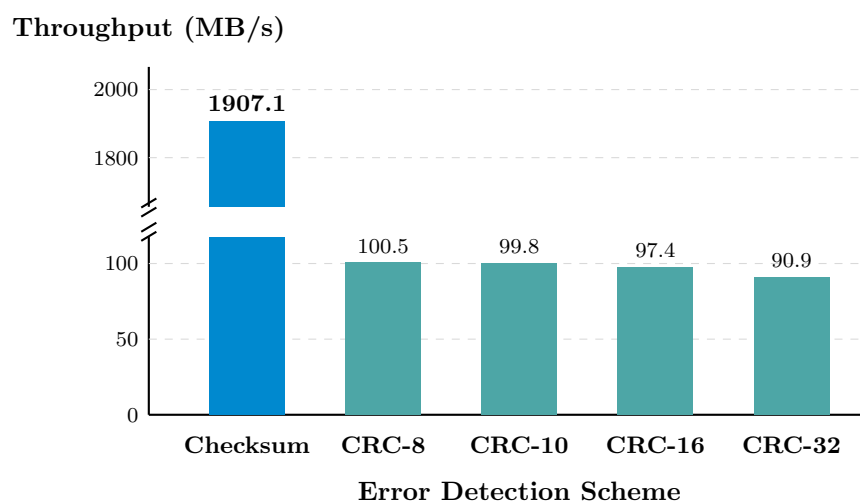


Figure 5: Computational throughput comparison demonstrating the dramatic performance advantage of word addition over bit-serial CRC (split Y-axis).

Analysis: The 16-Bit Checksum executes approximately **19–21× faster** than the bit-serial CRC algorithms. This performance disparity stems from underlying CPU architecture:

- **Word-level Accumulation:** The Internet Checksum processes 16 bits (2 bytes) per iteration using native CPU ALU addition instructions, taking only 30 word additions for a 60-byte frame.
- **Bit-serial Division:** The CRC routine requires 480 bitwise loop iterations, evaluating branch conditions, shifts, and XOR operations for every bit in the frame.

7 Concrete Boundary Collisions (Hex Dumps)

To demonstrate the theoretical boundary cases required by the assignment, real corrupted frames were captured from simulation runs:

7.1 Case 1: Error Detected by Both Schemes

A transmission corrupted by two flipped bits where both algorithms flag errors:

- **Original Checksum:** 0x5DEC → **Corrupted Checksum:** 0x5E10 (**Detected: YES**)

- **Original CRC-8: 0xFF → Corrupted CRC-8: 0x5C (Detected: YES)**

```
Original Frame Payload (44 Bytes):
54 68 65 20 62 72 6F 77 6E 20 66 6F 78 20 6A 75
6D 70 73 20 6F 76 65 72 20 74 68 65 20 6C 61 7A
79 20 64 6F 67 20 74 6F 64 61 79 2E
Corrupted Frame Payload (44 Bytes):
54 68 65 20 62 72 6F 77 6E 20 66 6B 78 20 6A 75
6D 70 73 20 6F 76 65 72 20 74 68 65 20 6C 61 7A
79 00 64 6F 67 20 74 6F 64 61 79 2E
Differing Bytes:
Byte offset 11: 0x6F ('o') -> 0x6B ('k') [XOR Mask: 0x04]
Byte offset 33: 0x20 (' ') -> 0x00 ('.') [XOR Mask: 0x20]
```

Listing 4: Case 1 Payload Hex Dumps and Differing Bytes

7.2 Case 2: Detected by Checksum, Missed by CRC-8 (CRC Collision)

A transmission where the error polynomial $E(x)$ happens to be an exact multiple of the CRC-8 generator $G(x)$, leaving the CRC remainder completely identical to the original, while the arithmetic Checksum catches the error:

- **Original Checksum: 0x5DEC → Corrupted Checksum: 0x5DFE (Detected: YES)**
- **Original CRC-8: 0xFF → Corrupted CRC-8: 0xFF (MISSED! CRC Collision)**

```
Original Frame Payload (44 Bytes):
54 68 65 20 62 72 6F 77 6E 20 66 6F 78 20 6A 75
6D 70 73 20 6F 76 65 72 20 74 68 65 20 6C 61 7A
79 20 64 6F 67 20 74 6F 64 61 79 2E
Corrupted Frame Payload (44 Bytes):
54 68 65 20 62 72 6F 77 6E 20 66 6D 78 20 6A 75
6D 70 73 20 6F 76 65 62 20 74 68 65 20 6C 61 7A
79 20 64 6F 67 20 74 6F 64 61 79 2E
Differing Bytes:
Byte offset 11: 0x6F ('o') -> 0x6D ('m') [XOR Mask: 0x02]
Byte offset 23: 0x72 ('r') -> 0x62 ('b') [XOR Mask: 0x10]
```

Listing 5: Case 2 Payload Hex Dumps (CRC-8 Collision)

7.3 Case 3: Detected by CRC-8, Missed by Checksum (Compensating Sum)

A transmission where an arithmetic increase in one byte is exactly cancelled by an equal arithmetic decrease in another byte at equivalent 16-bit word alignment:

- **Original Checksum: 0x5DEC → Corrupted Checksum: 0x5DEC (MISSED! Compensating Sum)**
- **Original CRC-8: 0xFF → Corrupted CRC-8: 0xBE (Detected: YES)**

```
Original Frame Payload (44 Bytes):
54 68 65 20 62 72 6F 77 6E 20 66 6F 78 20 6A 75
6D 70 73 20 6F 76 65 72 20 74 68 65 20 6C 61 7A
79 20 64 6F 67 20 74 6F 64 61 79 2E
Corrupted Frame Payload (44 Bytes):
54 68 65 20 62 72 6F 77 6E 20 66 6F 78 20 6A 75
6D 70 73 20 6F 76 65 72 20 74 68 65 20 68 61 7A
79 20 64 6F 67 20 74 6F 64 65 79 2E
Differing Bytes:
```

```
Byte offset 29: 0x6C ('l') -> 0x68 ('h') [Arithmetic Delta: -4]
Byte offset 41: 0x61 ('a') -> 0x65 ('e') [Arithmetic Delta: +4]
```

Listing 6: Case 3 Payload Hex Dumps (Compensating Checksum Collision)

Mathematical Explanation of Case 3: In the 60-byte verified frame, Byte 29 is located at serialized offset $16 + 29 = 45$ (the LSB of 16-bit word index 22), and Byte 41 is at serialized offset $16 + 41 = 57$ (the LSB of 16-bit word index 28). When Byte 29 decreases by 4 and Byte 41 increases by 4, the net delta to the 16-bit word sum is:

$$\Delta\text{Sum} = (-4) + (+4) = 0 \quad (11)$$

The Internet Checksum accumulator experiences zero total change, leaving the checksum completely oblivious to the corruption! Conversely, CRC-8 treats the bit flips algebraically as $E(x) = x^{242} \oplus x^{146}$. Since $E(x) \not\equiv 0 \pmod{G(x)}$, the CRC remainder changes to 0xBE, detecting the corruption immediately.

8 Testing Conditions, Limitations, and Enhancements

8.1 Ideal Testing Conditions vs. Real-World Environments

The experimental evaluations were conducted over the local loopback interface (127.0.0.1). Under these controlled conditions:

- Kernel socket buffers eliminate physical packet drops, packet truncation, and out-of-order delivery.
- Errors were introduced deterministically and stochastically via software routines.

In physical network media (such as Cat6 UTP copper cabling, optical fiber, or 802.11 Wi-Fi links), thermal noise introduces continuous background bit flips, while lightning or motor switches cause massive burst collisions. Real-world channels frequently suffer from damaged frame preambles, truncated headers, and variable frame sizes.

8.2 Known Limitations and Practical Improvements

1. **Table-Driven CRC Lookup:** The evaluated bit-serial CRC loop achieves ~ 100 MB/s. Production implementations (such as the Linux kernel network stack) precompute a 256-entry lookup table:

```
1   crc = (crc >> 8) ^ crc32_table[(crc ^ data[i]) & 0xFF];
2
```

This eliminates inner bitwise loops, processing one full byte per lookup and elevating throughput to several gigabytes per second.

2. **Hardware Acceleration via Dedicated CPU Instructions:** Modern x86 processors provide the `_mm_crc32` intrinsic (SSE4.2), and ARMv8 provides dedicated CRC32 instructions that compute IEEE 802.3 CRC at full memory bus saturation rates (> 20 GB/s).
3. **Integration with Automatic Repeat reQuest (ARQ):** In the current implementation, corrupted frames are flagged and discarded. Layering a Stop-and-Wait or Go-Back-N ARQ protocol with ACK/NACK signaling and retransmission timers would upgrade the error detection engine into a fully reliable transport protocol.

9 Conclusion

This investigation designed, implemented, and empirically evaluated the 16-bit Internet Checksum and four CRC polynomial standards across 1,000,000 simulated transmissions and 5,000,000 micro-benchmark executions.

The key conclusions of this study are:

1. **Single-Bit Errors:** Both the 16-bit Internet Checksum and all four CRC variants achieve strictly 100.0000% detection.
2. **Compensating Error Vulnerability:** The Internet Checksum fails on approximately 2.08% of double-bit errors due to compensating arithmetic cancellations in one's complement addition. In contrast, CRC-10, CRC-16, and CRC-32 achieve 100.0000% detection.
3. **Odd-Parity Protection:** By utilizing polynomials containing the $(x + 1)$ factor, all evaluated CRCs achieve 100.0000% detection of odd-weight error patterns, whereas the Checksum missed 0.063% of multi-bit patterns.
4. **Burst Error Bounding:** For burst lengths $L \leq r$, CRC detection is strictly 100.0000%. When $L > r$, the miss rate closely conforms to the theoretical $(1/2)^r$ bound.
5. **Engineering Tradeoff:** The 16-bit Checksum achieves 1907.1 MB/s ($\sim 20\times$ faster than bit-serial CRC), elucidating why software-driven Transport Layer protocols (TCP/UDP) prioritize arithmetic checksums for processing speed while hardware-accelerated Data Link Layer protocols (Ethernet IEEE 802.3) mandate CRC-32 for robust data integrity.

10 Sources Used

1. Braden, R., Borman, D., and Partridge, C., *Computing the Internet Checksum*, RFC 1071, Internet Engineering Task Force (IETF), 1988.
2. IEEE Standards Association, *IEEE 802.3-2022: IEEE Standard for Ethernet*, IEEE, 2022.
3. Kurose, J. F., and Ross, K. W., *Computer Networking: A Top-Down Approach*, 8th ed., Pearson, 2021.
4. Tanenbaum, A. S., and Wetherall, D. J., *Computer Networks*, 5th ed., Prentice Hall, 2011.
5. ITU-T Recommendation I.432.1, *B-ISDN User-Network Interface – Physical Layer Specification*, International Telecommunication Union, 1996.
6. AI Assistance (Gemini 3.1 Pro): AI was used for statistical sampling and benchmarking, generation of randomized test cases, production of statistical reports, and creation of latex graphs and visualizations. Since I am not yet familiar with latex formatting, the document typesetting and formatting was also handled with AI assistance.